DATA 266 – Generative AI and LLM

Name: Kavya Ayyappan sjsu_id: 016052738

1A. Fundamental architecture of a GAN

A GAN has two neural networks: a generator and a discriminator. They work against each other. The generator takes random noise (z) and makes fake images. The discriminator looks at real and fake images. It tries to tell which ones are real.

Training has two steps. First, we train the discriminator while keeping the generator fixed. It learns to spot fake images. Next, we keep the discriminator fixed and train the generator. The generator uses the discriminator's feedback to make better images. We repeat these steps many times.

The goal is to make fake images that look like real ones. Ideally, the generator learns the patterns in the real data. At that point, the discriminator cannot tell real and fake images apart. It would be right about 50% of the time.

1B. Mode collapse and ways to reduce it

Mode collapse happens when the generator keeps making the same kind of image. It does not show the full variety of the training data. For example, the dataset may have shirts, shoes, and bags. But the generator might only make shoes. This can happen when it finds a few images that fool the discriminator. It keeps making similar images instead of learning the rest. Training can also become unstable if one network gets much stronger than the other.

Technique 1 – Wasserstein GAN (WGAN): WGAN uses Wasserstein distance instead of the usual GAN loss. Its critic gives the generator more useful feedback, even when the images are poor. This helps training stay more stable. It may also reduce mode collapse, but it does not remove the problem completely.

Technique 2 – Gradient penalty (WGAN-GP): WGAN needs a critic that changes smoothly. The first WGAN used weight clipping to control the critic. But weight clipping can make learning harder. WGAN-GP uses a gradient penalty instead. This helps the generator get steady feedback. It may help the model learn more kinds of images.

Another method – Instance noise: We can add a little noise to both real and fake images. This stops the discriminator from becoming too sure of its answers. The generator then gets more useful feedback. This can make training steadier, but it cannot fully prevent mode collapse.

1C. Three GAN variants

1. Deep Convolutional GAN (DCGAN): A basic GAN does not make full use of how pixels are arranged in an image. DCGAN uses convolutional layers, which work well with images. Its generator uses transposed convolutions to turn a small code into a larger image. This helps it create images more effectively.

2. Conditional GAN (cGAN): A regular GAN makes images from random noise. We cannot directly choose what it makes. A cGAN adds a condition (y), such as a label or a

feature. Both networks receive this condition. For example, we can ask it to make a smiling face. This gives us more control over the image.

3. CycleGAN: Sometimes we have two groups of images, but no matching pairs. For example, we may have horse photos and zebra photos. CycleGAN can learn from these separate groups. It uses two generators to change images in both directions. It also uses two discriminators to check if the images look real. Cycle consistency means that changing an image and then changing it back should give us something close to the original. This helps keep the main content of the image.

2. FashionMNIST: FC-AE and VAE

Dataset and setup: FashionMNIST has grayscale clothing images. Each image is 28 × 28 pixels, or 784 values. I used 60,000 images for training and 10,000 for testing. I built both models using PyTorch. The encoder goes from 784 → 256 → 64 → 16, and the decoder goes back in reverse. The VAE also predicts a mean and log-variance for its 16-value latent space. I trained each model for 10 epochs. I used the Adam optimizer, a learning rate of 0.001, and a batch size of 128. The random seed was 2738.

Fully Connected Autoencoder (FC-AE): The encoder turns the 784 image values into a short code. The decoder uses that code to rebuild the image. During training, the model tries to make the rebuilt image as close to the original as possible. As we learned in class, an ordinary autoencoder is good at rebuilding images. But random points in its latent space may not create useful new images.

Variational Autoencoder (VAE): The VAE does not make only one fixed code for an image. Its encoder predicts a mean and a log-variance. It then samples a code using the reparameterization trick. The decoder uses this code to rebuild the image. The VAE loss has two parts: reconstruction loss and KL divergence. KL divergence helps keep the latent space smooth. This lets us sample new points and make new images.

Evaluation: I tested both models on the FashionMNIST test images. I compared their reconstruction BCE losses. I also looked at the original images next to the rebuilt images. For the VAE, I made new images by sampling random points from its latent space.

Experiment results: I trained both models for 10 epochs using seed 2738. The table shows the results from my latest notebook run. Loss values are averages per image.

Test results (loss per image)

Model	Reconstruction BCE	KL	Total loss	Observation
FC-AE	222.108	—	222.108	Lower reconstruction loss
VAE	232.821	12.000	244.820	Can generate new images

Results and discussion: The FC-AE had a lower reconstruction BCE than the VAE: 222.108 compared with 232.821. This means the FC-AE rebuilt the test images more closely based on this measure. The VAE also had a KL loss of 12.000. Its total loss was 244.820. The VAE has a higher reconstruction loss because it also learns a smooth latent space. This helps it make new clothing images from random points. Both models improved during training.





